

1. Мова SQL

SQL (Structured query language – мова структурованих запитів) – декларативна мова програмування для взаємодії користувача з базами даних, що застосовується для формування запитів, оновлення і керування реляційними БД, створення схеми бази даних і її модифікації, системи контролю за доступом до бази даних.

Сам по собі SQL не є ні системою керування базами даних, ні окремим програмним продуктом. Не являючись мовою програмування в тому розумінні, як C# або Java, SQL може формувати інтерактивні запити або, будучи вбудованою в прикладні програми, виступати в якості інструкцій для керування даними. Стандарт SQL, крім того, включає функції для визначення зміни, перевірки і захисту даних.

SQL – це діалогова мова програмування для здійснення запиту і внесення змін до бази даних, а також управління базами даних. Багато баз даних підтримує SQL з розширеннями до стандартної мови. Ядро SQL формує командна мова, яка дозволяє здійснювати пошук, вставку, оновлення, і вилучення даних, використовуючи систему управління і адміністративні функції. SQL також включає CLI (Call Level Interface) для доступу і управління базами даних дистанційно.

1.1. Історія мови SQL

Перша версія SQL була розроблена на початку 1970-х років у IBM. Ця версія мала назву SEQUEL і була призначена для обробки й пошуку даних, що містилися в реляційній базі даних IBM, System R. Мова SEQUEL пізніше була стандартизована Американським Держстандартом ANSI (American National Standards Institute) у 1986р. під назвою SQL (таблиця 1). Попри це, навіть англійські фахівці по-колишньому часто називають SQL [сіквел], замість [ес-кью-ел].

Таблиця 1 – Історія версій стандартів SQL

Рік	Назва	Інша назва	Опис
1986	SQL-86	SQL-87	Вперше оприлюднено ANSI. Ратифіковано ISO в 1987.
1989	SQL-89	FIPS 127-1	Невеликі зміни.
1992	SQL-92	SQL2	Вагомі зміни.
1999	SQL:1999	SQL3	Додано регулярні вирази, рекурсивні запити, тригери та деякі об'єктно-орієнтовані нововведення.
2003	SQL:2003	SQL 2003	Впроваджено розширення для роботи з XML-даними.
2006	SQL:2006	SQL 2006	ISO/IEC 9075-14:2006. Функціональність роботи з XML-даними значно розширено. З'явилась можливість сумісного використання в SQL та XQuery.
2008	SQL:2008	SQL 2008	Вдосконалено можливості віконних функцій, усунуто деякі неоднозначності стандарту SQL:2003. Легалізовано ORDER BY поза визначенням курсору. Додано тригери INSTEAD OF. Додано заяви TRUNCATE.
2011	SQL:2011	SQL 2011	ISO/IEC 9075:2011.

Варто відзначити, що SEQUEL не був єдиною мовою подібного призначення. У Каліфорнійському університеті в Берклі була розроблена некомерційна СКБД Ingres (що була, між іншим, дальнім прародичем популярної зараз некомерційної СКБД PostgreSQL), що була реляційною СКБД, але використовувала свою власну мову QUEL, яка, проте, в подальшому не витримала конкуренції з мовою SQL.

Першими СКБД, що підтримують нову мову SQL, стали в 1979 році «Oracle V2» для машин VAX від компанії Relational Software Inc. (що згодом стала компанією Oracle) і «System/38» від IBM, заснована на System/R.

Спочатку SQL розроблялась як мова запитів і управління даними, пізніші модифікації SQL створено розробниками комерційних СКБД, які додали процедурні конструкції, control-of-flow команди і розширення мов. З випуском стандарту SQL:1999 такі розширення були формально запозичені як частина мови SQL через Persistent Stored Modules (SQL/PSM).

Перший офіційний стандарт мови SQL був прийнятий ANSI в 1986 й ISO (Міжнародною організацією зі стандартизації) в 1987 (так званий SQL-86) і дещо уточнений в 1989 році. Подальший розвиток мови постачальниками СКБД привів до ухвалення в 1992 р. нового розширеного стандарту (ANSI SQL-92 або просто SQL2). Наступним стандартом став SQL:1999 (SQL3). Зараз діє стандарт, прийнятий в 2003 році (SQL:2003) з невеликими модифікаціями, внесеними пізніше.

1.2. Структура SQL

SQL (Structured query language – мова структурованих запитів), складається з:

- DDL (Data Definition Language) – робота із структурою бази,
- DML (Data Manipulation Language) – робота з даними,
- DCL (Data Control Language) – робота з правами,
- TCL (Transaction Control Language) – робота з транзакціями.

В кожній із складових SQL є ряд операторів (фраз), знання яких є базовим і обов'язковим для вивчення.

- **DDL (робота із структурою бази):**
 - **CREATE** – створення об'єкта (наприклад, таблиці);
 - **ALTER** – зміна об'єкта (наприклад, додавання/зміна полів таблиці);
 - **DROP** – видалення об'єкта.
- **DML (робота з даними):**
 - **SELECT** – вибірка строк;
 - **INSERT** – вставка;
 - **UPDATE** – зміна;
 - **DELETE** – видалення.
- **DCL (робота з правами):**
 - **GRANT** – надання прав користувачу;
 - **DENY** – явна заборона для користувача;
 - **REVOKE** – відміна заборони/дозволу користувачу.
- **TCL (робота з транзакціями):**
 - **BEGIN TRANSACTION** – почати транзакцію;
 - **COMMIT** – прийняти зміни прийняті в транзакції;
 - **ROLLBACK** – відкат.

Оскільки SQL не є мовою програмування (тобто не надає засобів для автоматизації операцій з даними), введені різними виробниками розширення стосувалися в першу чергу процедурних розширень. Це збережені процедури (англ. stored procedures) і процедурні мови-«надбудови». Практично у кожній СКБД застосовується своя процедурна мова. Такі мови для найпопулярніших СКБД приведені в таблиці 2.

Таблиця 2 – Процедурні мови (діалекти) SQL

СКБД	Коротка назва	Розшифровка
Microsoft SQL Server/ Sybase ASE	Transact-SQL	Transact-SQL
MySQL	SQL/PSM	SQL/Persistent Stored Module
Oracle	PL/SQL	Procedural Language/SQL (заснований на мові Ada)
PostgreSQL	PL/pgSQL	Procedural Language/PostgreSQL Structured Query Language (схожий на Oracle PL/SQL)
Borland InterBase/ Firebird	PSQL	Procedural SQL
IBM DB2	SQL PL	SQL Procedural Language (розширює SQL/PSM)

Крім цього, можна виділити оператори, які є складовими компонентами заяв і запитів, показані у таблиці 3.

Таблиця 3 – Оператори SQL

Оператор	Опис	Приклад
=	Рівно	Author = 'Alcott'
<> or !=	Не рівно	Dept <> 'Sales'
>	Більше	Hire_Date > '2012-01-31'
<	Менше	Bonus < 50000.00
>=	Більше рівно	Dependents >= 2
<=	Менше рівно	Rate <= 0.05
BETWEEN	Між діапазоном, включно	Cost BETWEEN 100.00 AND 500.00
LIKE	Збіг з шаблоном символів	First_Name LIKE 'Will%'
IN	Рівно одному з множини вказаних	DeptCode IN (101, 103, 209)
IS or IS NOT	Перевірка на наявність (не пуста)	Address IS NOT NULL

Далі буде розглянуто вказані ключові оператори SQL та особливості й умови їх використання.

Оператор CREATE TABLE

Існує кілька операторів SQL для керування метаданими, використовуваних для створення, зміни або видалення баз даних чи об'єктів, що зберігаються в них (таблиць, строк й ін.). Розглянемо деякі з них: CREATE TABLE, ALTER TABLE й DROP.

Для створення нової таблиці необхідно використовувати оператор CREATE TABLE, синтаксис якого має вигляд:

```
CREATE TABLE table ( column1 type1 [(size1)][CONSTRAINT _ column-constraint1] [,
column2 type2 [(size2)][CONSTRAINT _ column-constraint2] [, ...] [CONSTRAINT table-
constraint1 _ [table-constraint2 [, ...]]];
```

У цьому операторі слід вказати ім'я поля, тип даних для нього (тип даних повинен підтримуватися даною СКБД), довжину (для деяких типів полів) і, якщо потрібно, серверні обмеження (із застосуванням ключового слова CONSTRAINT). Наприклад, наступний запит створює таблицю з ім'ям Simple із чотирма колонками – LastName, FirstName, EMail й

HomePage:

```
CREATE TABLE Simple (FirstName varchar(50) NOT NULL, LastName varchar(50) NOT NULL, EMail varchar(50), HomePage varchar(255) )
```

Ми можемо розширити цю таблицю додаванням поля Personi, що буде використане як первинний ключ:

```
CREATE TABLE Simple ( Personi Integer NOT NULL PRIMARY KEY, FirstName varchar(50) NOT NULL, LastName varchar(50) NOT NULL, EMail varchar(50), HomePage varchar(255) )
```

і вказати, що комбінація полів LastName й FirstName повинна бути унікальна:

```
CREATE TABLE Simple ( Personi Integer NOT NULL PRIMARY KEY, FirstName varchar(50) NOT NULL, LastName varchar(50) NOT NULL, EMail varchar(50), HomePage varchar(255), CONSTRAINT SimpleConstraint UNIQUE (FirstName, LastName) )
```

Використовуючи оператор SELECT і ключове слово INTO, ми можемо створювати нові таблиці, засновані на умові, зазначеній в операторі WHERE. Наприклад:

```
SELECT * INTO NewOrders FROM Orders WHERE OrderDate > 1/1/14
```

Цей запит створить нову таблицю NewOrders і заповнить її даними про замовлення, починаючи з 1 січня 2014 року.

Оператор ALTER TABLE

Для зміни структури існуючої таблиці можна використати оператор ALTER TABLE. Застосовуючи його, можна додати або видалити поле або серверне обмеження. Існує чотири різновиди оператора ALTER TABLE.

Перший різновид цього оператора використовується для додавання стовпчика до таблиці, і її синтаксис має вигляд:

```
ALTER TABLE table ADD [COLUMN] column datatype [(size)] [CONSTRAINT single-column-constraint]
```

У запитах такого виду визначається ім'я таблиці, ім'я нового поля, його тип даних і, якщо потрібно, розмір. Крім цього, можна вказати серверне обмеження, пов'язане з даним полем. Наприклад, для до поля Phone з таблиці Simple, створеної раніше, можна виконати наступний запит:

```
ALTER TABLE Simple ADD Phone varchar(30)
```

Другий різновид оператора ALTER TABLE застосовується для додавання серверних обмежень до таблиці, а її синтаксис має вигляд:

```
ALTER TABLE table ADD CONSTRAINT constraint
```

Такі запити дозволяють додавати тільки індекси, які дозволяють використовувати відповідні поля в якості первинних або зовнішніх ключів.

Третій різновид оператора ALTER TABLE застосовується для видалення поля з таблиці:

```
ALTER TABLE table DROP [COLUMN] column
```

Ключове слово COLUMN використовувати не обов'язково. Наприклад:

```
ALTER TABLE Simple DROP Phone
```

Зверніть увагу на те, що для видалення проіндексованих полів варто спочатку видалити індекс.

Це можна зробити за допомогою **четвертого різновиду** оператора ALTER TABLE:

```
ALTER TABLE table DROP CONSTRAINT index
```

Нижче наведений приклад такого запиту:

```
ALTER TABLE Simple DROP CONSTRAINT PrimaryKey
```

Оператор DROP

Для видалення таблиць або індексів можна використати оператор DROP, що має два різновиди. **Перший різновид** застосовується для видалення таблиці з бази даних:

DROP TABLE table

Другий різновид використовується для видалення індексу:

DROP INDEX index ON table

Оператор SELECT

Вибір даних являє собою операцію, що найбільш часто зустрічається у SQL-запитах. **Оператор SELECT – один з найважливіших операторів цієї мови**, застосовуваний для вибору даних. Синтаксис цього оператора має такий вигляд:

SELECT column-list FROM table-list [WHERE where-clause] [ORDER BY order-by-clause]

Оператори SELECT повинні містити слова SELECT й FROM; інші ключові слова, такі як WHERE або ORDER BY, є необов'язковими.

За ключовим словом SELECT ідуть відомості про те, які саме поля необхідно включити в результуючий набір даних. Зірочка (*) позначає всі поля таблиці, наприклад:

SELECT *

Для вибору одного стовпчика застосовується наступний синтаксис:

SELECT CompanyName

Приклад вибору декількох стовпчиків має вигляд:

SELECT CompanyName, ContactName, ContactTitle

Якщо вибір даних здійснюється з декількох таблиць і при цьому вибираються однойменні поля з різних таблиць, варто посилатися на імена таблиць для повної ідентифікації полів, що включають у результуючий набір даних, наприклад:

SELECT Customers.CompanyName, Shippers.CompanyName

Оператор FROM

Для вказування імен таблиць, з яких вибираються записи, застосовується ключове слово FROM, наприклад:

SELECT * FROM Customers

Цей запит поверне всі поля з таблиці Customers.

Якщо в результуючому наборі даних потрібні тільки поля CompanyName й ContactName, ми можемо ввести наступний вираз з SELECT:

SELECT CompanyName, ContactName FROM Customers

Приклад запиту до більш ніж однієї таблиці наведений нижче:

SELECT Customers.CompanyName, Shippers.CompanyName FROM Customers, Shippers

Оператор WHERE

Для фільтрації результатів, що повертаються оператором SELECT, можна використати оператор WHERE, синтаксис якого має вигляд:

WHERE expression1 [{AND | OR} expression2 [...]]

Наприклад, замість одержання повного списку продуктів можна обмежитися тільки тими з них, у яких значення поля Categoryi дорівнює 4:

SELECT * FROM Products WHERE Categoryi = 4

У операторі WHERE можна використати різні вирази, наприклад:

SELECT * FROM Products WHERE Categoryi = 2 AND Supplieri > 10

або:

SELECT ProductName, UnitPrice FROM Products WHERE Categoryi = 3 OR UnitPrice < 50

або:

SELECT ProductName, UnitPrice FROM Products WHERE Discontinued IS NOT NULL

Виразення 'IS NOT NULL' означає, що відповідний стовпчик результуючого набору

даних не може містити порожніх значень. У пропозиції WHERE можна використати один із шести операторів відношення, визначених у SQL. Ці оператори наведені в таблиці 4.

Таблиця 4 – Шість операторів відношення у SQL

Оператор	Опис
<	Менше
<=	Менше або дорівнює
<>	Не дорівнює
=	Дорівнює
>	Більше
>=	Більше або дорівнює

Крім перерахованих вище простих операторів порівняння, можна використати й спеціальні оператори порівняння, наведені в таблиці 5.

Таблиця 5 – П'ять операторів відношення у SQL

Оператор	Опис
ALL	Застосовується разом з операторами порівняння при порівнянні зі списком значень
ANY	Застосовується разом з операторами порівняння при порівнянні зі списком значень
BETWEEN	Застосовується при перевірці знаходження значення усередині заданого інтервалу (включаючи його границі)
IN	Застосовується для перевірки наявності значення в списку
LIKE	Застосовується при перевірці відповідності значення заданій масці

Приведемо кілька прикладів застосування цих операторів. Для зіставлення даних з маскою застосовується ключове слово LIKE:

```
SELECT CompanyName, ContactName FROM Customers WHERE CompanyName LIKE 'M%'
```

У даній масці символ '%' (відсоток) заміняє будь-яку послідовність символів, а символ '_' (підкреслення) – один будь-який символ. Той же самий результат може бути отриманий у такий спосіб:

```
SELECT CompanyName, ContactName FROM Customers WHERE CompanyName BETWEEN 'M' AND 'N'
```

В останньому прикладі ми можемо розширити область пошуку. Зокрема, при пошуку компаній з іменами, що починаються з букв від А до С, можна виконати наступний оператор SELECT:

```
SELECT CompanyName, ContactName FROM Customers WHERE CompanyName BETWEEN 'A' AND 'D'
```

Використовуючи оператор LIKE, ми можемо звужити діапазон пошуку, застосувавши більш складну маску для порівняння. Наприклад, щоб знайти компанії, що містять у своїй назві підстроку bl, можна застосувати наступний запит:

```
SELECT CompanyName, ContactName FROM Customers WHERE CompanyName LIKE '%bl%'
```

Маска '%bl%' показує, що до й після шуканої підстроки може бути будь-яка кількість довільних символів.

Використовуючи оператор IN, можна задати список значень, у якому повинне втримуватися значення поля:

```
SELECT CompanyName, ContactName FROM Customers WHERE Customeri IN
```

('ALFKI', 'BERGS', 'VINET')

Оператори AND, OR й NOT

Ми вже розглядали приклад застосування оператора AND для логічних операцій, пов'язаних з вимогою, щоб запис задовольняв двом різним критеріям. Розглянемо наступний запит:

```
SELECT CompanyName, ContactName FROM Customers WHERE CompanyName LIKE 'S%' AND Country = 'Ukraine'
```

Результатом виконання цього запиту буде список замовників, що перебувають в Україні, назва яких починається з букви S.

Оператор OR дозволяє вибрати записи, що задовольняють хоча б одній з перерахованих умов, у той час як оператор NOT використовується для виключення з набору записів, що задовольняють даній умові. Наприклад, можна застосувати оператор OR для пошуку всіх замовників, що перебувають в Україні, або що носять назву, яка починається з букви S (і при цьому знаходяться де завгодно):

```
SELECT CompanyName, ContactName FROM Customers WHERE CompanyName LIKE 'S%' OR Region='Ukraine'
```

У цьому випадку результуючий набір даних буде містити записи, у яких значення поля CompanyName задовольняє першій умові, плюс всі записи, у яких значення поля Region задовольняє другій умові.

Тепер розглянемо приклад застосування оператора NOT. Для виключення деяких замовників з результуючого набору даних можна використати запит виду:

```
SELECT CompanyName, ContactName FROM Customers WHERE Country NOT IN ('Ukraine', 'Poland')
```

У результаті виконання цього запиту ми одержимо список замовників із всіх країн, крім України й Польщі.

Оператор ORDER BY

Оператор ORDER BY (необов'язковий) застосовується для сортування результуючого набору даних по одному або декількох стовпчиках. Для визначення порядку сортування використовуються ключові слова: ASC (по зростанню) або DESC (по убутанню). За замовчуванням дані сортуються по зростанню. Синтаксис оператора ORDER BY має вигляд:

```
ORDER BY column1 [{ASC | DESC}] [, column2 [{ASC | DESC}] [...]
```

Наприклад, для сортування співробітників по прізвищу й потім по імені варто використати наступний SQL-запит:

```
SELECT LastName, FirstName, Title FROM Employees ORDER BY LastName, FirstName
```

Якщо сортування даних потрібно в убутуючому порядку (наприклад, потрібен список продуктів у порядку убутання цін), використовується ключове слово DESC:

```
SELECT ProductName, UnitPrice FROM Products ORDER BY UnitPrice DESC
```

Оператор JOIN (зв'язування таблиць)

Як ми вже переконалися, можна створювати запити, що дозволяють витягати дані з декількох таблиць. Одна з можливостей зробити це полягає у зв'язуванні таблиць по одному або декількох полях. Зверніть увагу на те, що без зв'язування таблиць у результаті вийде комбінаторний запит – набір даних, що містить всі можливі комбінації рядків кожної з вихідних таблиць (відомий також як декартовий добуток):

```
SELECT ProductName, CategoryName FROM Products, Categories
```

у той час як запит, показаний нижче, приводить до відображення списку продуктів із

вказівкою, до якої категорії належить даний продукт:

```
SELECT ProductName, CategoryName FROM Products, Categories WHERE Products.Categoryi = Categories.Categoryi
```

Можна зрівняти результати цих двох запитів. У загальному випадку синтаксис для зв'язування таблиць має вигляд:

```
SELECT column-list FROM table1, table2 WHERE table1.column1=table2.column2
```

Наступні кілька прикладів зв'язування таблиць характерні для Microsoft Access й Microsoft SQL Server і можуть не працювати з іншими СКБД, однак ілюструєма ними функціональність досить важлива.

Існує кілька типів зв'язування таблиць. Наприклад, оператор SQL **inner join** здійснює так зване внутрішнє з'єднання таблиць – у цьому випадку в результуючому наборі даних будуть записи, у яких значення в зв'язаних полях збігаються:

```
SELECT ProductName, CategoryName FROM Products INNER JOIN Categories ON Products.Categoryi = Categories.Categoryi
```

Так звані зовнішні з'єднання (**outer joins**) дозволяють включити в результат запиту всі рядки з однієї таблиці й відповідні їм рядки з іншої таблиці. Наприклад:

```
SELECT ProductName, CategoryName FROM Products LEFT OUTER JOIN Categories ON Products.Categoryi = Categories.Categoryi
```

Це було так зване ліве зовнішнє з'єднання (**left outer join**). Існують також праві зовнішні з'єднання (**right outer join**), що повертають всі рядки із другої (тобто правої) таблиці й відповідні їм рядки з іншої таблиці:

```
SELECT ProductName, CategoryName FROM Products RIGHT OUTER JOIN Categories ON Products.Categoryi = Categories.Categoryi
```

Комбінуючи ліве й праве зовнішні з'єднання, можна одержати повне зовнішнє з'єднання, що повертає всі дані з обох таблиць:

```
SELECT ProductName, CategoryName FROM Products FULL OUTER JOIN Categories ON Products.Categoryi = Categories.Categoryi
```

Для одержання всіх комбінацій рядків з обох таблиць (декартового добутку) можна використати ключове слово **CROSS JOIN** без вказівки полів, що зв'язують:

```
SELECT ProductName, CategoryName FROM Products CROSS JOIN Categories
```

Якщо в запиті використовується більше трьох таблиць, можна використовувати вкладені з'єднання.

Оператор GROUP BY

Для обчислення сумарних значень на основі даних однієї або декількох таблиць можна використати оператор **GROUP BY**, що має такий синтаксис:

```
GROUP BY {column1} [, ...]
```

Наприклад, наступний запит зв'яже дві таблиці, сортує їх по полю **Customeri**, для кожного значення **Customeri** створює один рядок у результуючому наборі даних й обчислює кількість значень поля **Orderi** для кожного значення **Customeri**:

```
SELECT Customers.Customeri, COUNT (Orders.Orderi) FROM Customers INNER JOIN Orders ON Customers.Customeri = Orders.Customeri GROUP BY Customers.Customeri
```

У наведеному вище прикладі запиту використано в операторі **SELECT** агрегатну функцію **COUNT**, що обчислює кількість значень.

Оператор HAVING

Оператор **HAVING** має призначення, подібне до оператора **WHERE**, але використовується з агрегатними даними. Наприклад:

```
SELECT Customers.Customeri, COUNT (Orders.Orderi) FROM Customers INNER JOIN Orders ON Customers.Customeri = Orders.Customeri GROUP BY Customers.Customeri HAVING COUNT(Orders.Orderi) >= 10
```

Цей запит аналогічний попередньому, але в результуючий набір даних включені тільки ті замовники, які розмістили десять або більше замовлень.

Оператори ALL та DISTINCT

До цього моменту ми розглядали, як витягти всі або задані стовпчики з однієї або декількох таблиць. Для керування виводом рядків результуючого набору даних, що дублюються, можна використати ключові слова ALL або DISTINCT у виразі SELECT. Ключове слово DISTINCT указує, що рядки результуючого набору даних повинні бути унікальні, тоді як ключове слово ALL указує, що повертати треба всі рядки. Наприклад, для витягу назв країн, у яких є замовники, можна використати наступний запит:

```
SELECT DISTINCT Country FROM Customers
```

Відзначимо, що ключове слово ALL використовується по замовчуванню. Якщо в запиті потрібно вивести більше одного стовпчика й при цьому використане слово DISTINCT, то результуючий набір даних буде містити різні рядки, але деякі значення того самого поля в різних рядках можуть збігатися.

Оператор TOP

Ключове слово TOP може бути використане для повернення перших n рядків або перших n відсотків таблиці. Наприклад, запит:

```
SELECT TOP 10 * FROM PRODUCTS ORDER BY ProductName
```

повертає перші 10 продуктів з таблиці, тоді як запит:

```
SELECT TOP 25 PERCENT * FROM PRODUCTS ORDER BY ProductName
```

поверне першу чверть (25%) записів таблиці.

Оператор UPDATE

Дотепер розглядалися оператори SQL для витягу даних. Крім цього, мова SQL може бути використана для відновлення й видалення даних, копіювання записів в інші таблиці й виконання багатьох інших операцій. Нижче розглянуто оператори UPDATE, DELETE й INSERT, використовувані для рішення деяких із цих завдань.

Для зміни значень в одній або декількох колонках таблиці застосовується оператор UPDATE. Синтаксис цього оператора має вигляд:

```
UPDATE table SET column1 = expression1 [, column2 = expression2] [...] [WHERE criteria]
```

Вираз в операторі SET може бути константою або результатом обчислень. Наприклад, для підвищення цін на 5% всіх продуктів, що коштують менше 10 грн., можна виконати наступний запит:

```
UPDATE Products SET UnitPrice = UnitPrice * 1.05 WHERE UnitPrice < 10
```

Оператор DELETE

Для видалення рядків з таблиць варто використати оператор DELETE, синтаксис якого має вигляд:

```
DELETE FROM table [WHERE criteria]
```

Увага! Оператор WHERE не є обов'язковим, але якщо ви забудете його включити, з таблиці будуть вилучені всі записи.

Наприклад, для видалення зі списку всіх продуктів, які більше не поставляються, можна виконати наступний запит:

```
DELETE FROM Products WHERE Discontinued = 1
```

Відзначимо, що корисно використовувати оператор SELECT з тим же синтаксисом, що й оператор DELETE, щоб перевірити, які саме записи будуть вилучені, перш ніж дійсно їх видаляти (видалення з перевіркою). Нижче показаний оператор SELECT для наведеного вище запиту на видалення даних:

```
SELECT ProductName FROM Products WHERE Discontinued = 1
```

Можна використати в операторі WHERE більш складний критерій для визначення того, які записи повинні бути вилучені. Припустимо, нам потрібно видалити зі списку клієнтів тих із них, хто не мав замовлень до певної дати. Спочатку для цього варто виконати наступний SELECT, щоб визначити, що саме ми видаляємо:

```
SELECT CompanyName FROM Customers WHERE Customers.CustomerId NOT IN  
(SELECT CustomerId FROM Orders WHERE OrderDate > 01/01/14)
```

а потім замінити оператор SELECT на оператор DELETE:

```
DELETE FROM Customers WHERE Customers.CustomerId NOT IN (SELECT CustomerId  
FROM Orders WHERE OrderDate > 01/01/14)
```

Зауваження. При використанні в операторах SQL дати або часу, а також полів, що містять такі дані, слід уточнити синтаксис таких операторів у документації з комплекту поставки використовуваної СКБД.

Оператор INSERT

Для додавання записів у таблиці варто використовувати оператор INSERT, синтаксис якого має вигляд:

```
INSERT [INTO] table ( [column_list] { VALUES ( { DEFAULT | NULL | expression } } [, ...] )
```

Наприклад, для додавання нового клієнта в таблицю Customers можна використати наступний запит:

```
INSERT INTO Customers (CustomerId, CompanyName) VALUES ('XYZFO', 'XYZ Deli')
```

Базові операції над множинами

Операції з множинами поєднують в один результати декількох запитів. У таблиці 6 в порядку убування старшинства приводяться SQL-операції над множинами.

Таблиця 5 – SQL-операції над множинами

Операція	Виконувана функція	Приклад
UNION	Поєднує запити для видачі всіх рядків, що розрізняються кожного окремого запиту	...SELECT... UNION SELECT...;
INTERSECT	Поєднує запити для видачі всіх рядків, що розрізняються, обох запитів	...SELECT INTERSECT SELECT...;
MINUS	Поєднує запити для видачі всіх рядків, що розрізняються, виданих першим запитом, не другим.	...SELECT... MINUS SELECT...;

Зауваження: Якщо у двох таблицях зустрічаються записи, що дублюються, то при використанні множинних операцій UNION, MINUS, INTERSECT видається тільки один такий запис.

Рисунок 1 графічно ілюструє результати операцій з множинами. Затемнені частини показують результат, що повертається.

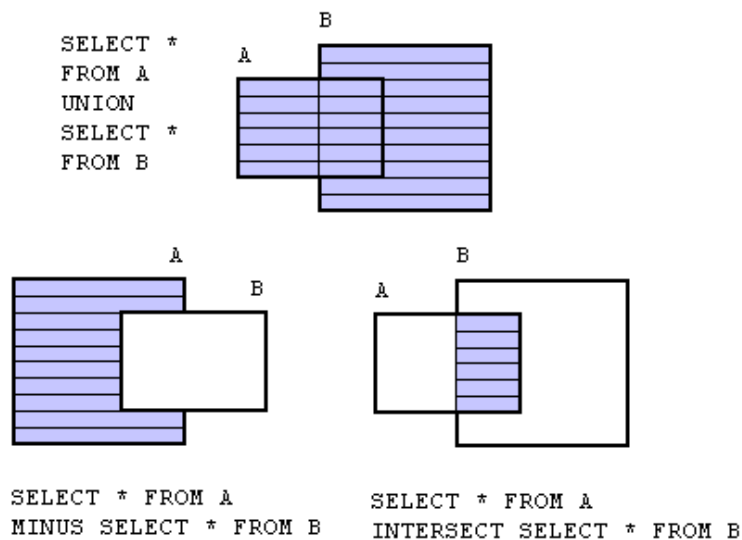


Рисунок 1 – Ілюстрація результатів операцій над множинами

Інші операції над множинами

У таблиці 6 описуються інші операції мови SQL над множинами.

Таблиця 6 – SQL-операції над множинами (2)

Операція	Виконувана функція	Приклад
(+)	Указує, що попередній стовпець – стовпець зовнішнього об'єднання	WHERE DEPT.DEPTNO=EMP.DEPT NO(+)
[table.]*	Вибирає в запиті всі стовпці з таблиць. Якщо починається ім'ям таблиці із крапкою, запит вибирає всі стовпці цієї таблиці.	SELECT * FROM EMP SELECT EMP.* FROM EMP
COUNT(expr)	Повертає число рядків, у яких expr не null	SELECT COUNT(COMM) FROM EMP
COUNT(*)	Повертає число рядків у таблиці, включаючи ті що з null	SELECT COUNT(*) FROM EMP
ALL	Повертає значення, що дублюються, у запитах й агрегатах	SELECT ALL *COUNT(ALL DEPTNO)
DISTINCT	Видаляє рядки, що дублюються, із запиту або видаляє значення, що дублюються, з агрегатного виразу	DELECT DISTINCT *... ...COUNT(DISTINCT DEPTNO)...
PRIOR	Визначає відношення "батько-син" у деревоподібному запиті. Вираз у лівій частині умови (EMPNO) представляє батьківський рядок; вираз справа (MGR) представляє рядок сина. Може використовуватися в будь-якому операторі деревоподібного запиту, а не тільки в CONNECT BY	CONNECT BY PRIOR EMPNO=MGR

2. Мова QBE

Запит являє собою спеціальним чином описану вимогу, яка визначає склад операцій, що проводяться над БД операцій по вибору, видаленню та модифікації збережених даних.

Як вже згадувалось, для підготовки запитів за допомогою різноманітних СКБД частіше за все використовуються 2 мови опису запитів.

- Мова QBE – “запити за зразком”
- SQL – “структурована мова запитів”

Головна відмінність між ними полягає в засобах формування запитів: мова QBE полягає в засобах формування запитів: мова QBE припускає ручне або візуальне формування запиту.

2.1. Характеристика мови QBE

Мова запитів за зразком QBE дозволяє задавати складні запити до БД, шляхом заповнення пропонуємої СКБД запитової форми. Такий спосіб задання запитів забезпечує наочність та не потребує вказівки алгоритму виконання операції – досить описати зразок очікуваного результату.

На мові QBE можна задавати запити **одно таблицні** та **багатотабличні** (які вибирають або обробляють дані з декількох зв’язаних таблиць).

За допомогою запитів на мові QBE можна виконувати наступні основні операції:

- Вибірку даних;
- Обчислення над даними;
- Вставку нових записів;
- Видалення записів;
- Модифікацію (зміну) даних.

Результатом виконання запита є нова таблиця, що називається відповідною (від слова “відповідь” – перші 2 операції), або поновлена вихідна таблиця (інші операції).

Вибір, вставка, видалення та модифікація можуть провадитися безумовно або у відповідність з умовами, що задаються за допомогою логічних виразів. Обчислення над даними задаються за допомогою арифметичних виразів і породжують у відповідних таблицях нові поля, що називаються **обчислюваними**.

Запитна форма має вигляд таблиці, ім’я та назва полів якої, співпадає з іменем та назвами полів відповідної вихідної таблиці. Щоб взяти імена доступних таблиць БД, в мові QBE передбачений запит на вибірку імен таблиць. Назви таблиць вихідної таблиці можуть вводитись в шаблон вручну або автоматично.

Для прикладу візьмемо таблиці БД, яка відноситься до торгової сфери та використовується в фірмі, що продає товари кількох видів. База даних включає в себе 4 наступні таблиці:

- EMP (службовці): ПІБ – прізвище, ініціали службовця, ЗАРПЛАТА – розмір посадового окладу, КЕРІВНИК – Прізвище та ініціали керівника, ВІДДІЛ – назва відділу, в якому працює службовець;
- SALES (продажі): ВІДДІЛ – назва відділу, ТОВАР – назва товару;
- SUPPLY(поставки): ТОВАР – назва товару, який постачається, ПОСТАЧАЛЬНИК – назва організації, яка постачає товар;
- TYPE (типи товарів): ТОВАР – назва товару, КОЛІР – його колір, ВАРТІСТЬ – вартість товару.

Для опису змінних в умовах відбору записів, а також для зв’язування шаблонів в записах використовуються елементи прикладу.

Елемент прикладу грає роль ідентифікатора змінної (як в мові програмування) та задається за допомогою символьно-цифрової послідовності. Елементи прикладу в шаблонах виділятимемо підкреслюванням. Вигляд (довжина та склад) елементу прикладу ролі не

грають: головне, щоб при використанні в декількох місцях шаблону він був однаковий. Таким чином, як елементи прикладу, зокрема, можна використовувати ідентифікатори **example**, **x** або **y**.

Для вказівки системі на необхідність включення у відповідну таблицю того чи іншого поля використовується “Р.”, що означає “надрукувати”.

Приклад 1. Запит на вибірку

Враховуючи сказане, запит на вибірку всіх зелених товарів можна записати в такому вигляді:

TYPE	ТОВАР	КОЛІР	ВАРТІСТЬ
	P.XX	зелений	

Словесно запит можна сформулювати таким чином: “Вивести товари **XX**, колір яких зелений”. Пусті колонки можна видаляти із запису.

Приклад 2. Видалення колонок

У нас не використовувані стовпчиком є ВАРТІСТЬ. Виходячи з цього, для наведеного шаблону можна записати такий еквівалентний шаблон

TYPE	ТОВАР	КОЛІР
	P.XX	зелений

Після заповнення шаблону для отримання результату нажимаємо відповідну клавішу, наприклад, <Enter> та починаємо виконання запиту. Результатом виконання вищенаведеного запиту буде таблиця.

TYPE	ТОВАР
	ручка
	чорнило

Приклад 3. Проста вибірка

Прикладом простої вибірки є запит: “Вивести всі можливі кольори товарів з таблиці TYPE”. Заповнений шаблон матиме вигляд:

TYPE	ТОВАР	КОЛІР	ВАРТІСТЬ
		Р.	

Таблиця відповіді має один стовпчик КОЛІР, в якому містяться значення: білий, червоний, пусто (значення не задане), зелений, синій. Дублюємі значення при цьому пропадають.

Якщо потрібно вивести дані з кількох полів вихідної таблиці, в кожному з відповідних стовпчиків шаблону записуємо “Р.”. Занесення “Р.” у всі стовпчики шаблону можна замінити записом “Р” в першому стовпчику шаблону під іменем таблиці.

Проста вибірка з упорядкуванням. Для упорядкування вибраних значень по зростанню та спаданню використовуються конструкції “АО.” Та “ДО.” відповідно. Якщо потрібно виконати по декількох стовпчиках, застосовуються конструкції “АО(1).”, “АО(2).”

Вибірка з кваліфікаторами (умовами). Вибір записів з вихідної таблиці в загальному випадку може бути оснований на: точному співпаданні, частковому співпаданні, порівнянні.

1. **Точне співпадання** задається введенням констант у відповідних полях шаблону, як у

випадку запита по товарах зеленого кольору.

2. **Часткове співпадання** задається за допомогою елементів прикладу. Зокрема для формулювання запиту про вивід всіх видів товарів, назви яких починаються з літери "і" можна скористатись конструкцією "P.ike", записаної в поле ТОВАР таблиці TYPE. В цьому випадку "P." задає вивід, "і"- константа, "ke" – елемент прикладу, який відіграє роль змінної. Використовуючи елементи прикладу, можна задавати різноманітні варіанти часткового співпадання із значеннями даних із таблиць: на початку "ike", в кінці "ха", всередині "x1ox2". Оскільки елементу прикладу відповідає будь-який символ, а також пустий (відсутність символу), то умові часткового співпадання "x1ox2" відповідають слова, які мають символ "о" не тільки всередині, але й на початку та в кінці.

Приклад 4. Часткове співпадання

Шаблон запиту з вибором товарів синього кольору, всередині назви яких є буква "р", виглядає таким чином,

TYPE	ТОВАР	КОЛІР	ВАРТІСТЬ
	P. <u>xpy</u>	синій	

Результат в цьому випадку буде таким:

TYPE	ТОВАР
	олівець
	чорнило

3. **Умови порівняння** записується за допомогою операцій порівняння: дорівнює (=), більше (>), менше (<), більше або дорівнює (>=) менше або дорівнює (<=), не дорівнює(!=), не більше (!>), не менше (!<).

Приклад 5. Умови порівняння

Запит імен співробітників, що працюють у відділі іграшок та отримують зарплату більше 360, виглядає таким чином

ЕМР	ПІБ	Зарплата	Керівник	Відділ
	P.	>360		іграшки

Результатом запиту є таблиця вигляду:

ЕМР	ПІБ
	Григор'єв А.Н.
	Томілов А.Н.

Приклад 6. Порівняння з елементами прикладу

Створимо шаблон запиту вибору імен та зарплат співробітників, які отримують більше ніж Левін П.Г. Інакше запит можна сформулювати так: "Нехай Левін П.Г. отримує зарплату в кількості s. Знайти всіх співробітників, які отримують зарплату більше, ніж s, та вивести їх зарплати. Порядок рядків в шаблоні значення не має.

EMP	ПІБ	Зарплата
	Р.	$P > \underline{s}$
	Левін П.Г.	$\underline{s}$

Співробітників, що отримують зарплату більшу, ніж Левін П.Г. в таблиці EMP не виявилось. В таких випадках результуюча таблиця виявляється порожньою.

EMP	ПІБ	Зарплата

Умови в запиті можуть задаватись по одному або декільком стовпчиках.

Приклад 7. Об'єднання умов

Для формулювання запиту вибірки імен та зарплат службовців, що отримують більше, ніж Левін П.Г. та працюють у відділі іграшок, достатньо в попередню запитну форму в перший рядок стовпчика ВІДДІЛ вставити слово "іграшки".

Запит виду: "Знайти імена та зарплати службовців, які отримують зарплату більшу, ніж Белкін Б.Н., та працюють у відділі, що продає ручки", виглядає таким чином:

EMP	ПІБ	Зарплата	Керівник	Відділ
	Р.	$P > \underline{s}$		<u>department</u>
	Белкін Б.Н.	$\underline{s}$		

SALES	Відділ	Товар
	<u>department</u>	ручка

Результатом виконання цього запиту буде таблиця вигляду:

EMP	ПІБ	Зарплата
	Левін П.Г.	440
	Сьомін С.В.	440

Тут елемент прикладу department зв'язує дві вихідні таблиці по полю ВІДДІЛ, а елемент прикладу s використовується для зв'язування умов вибору в рамках однієї вихідної таблиці EMP.

Приклад 8. Запит в шаблоні якого 2 зв'язки

Нехай потрібно знайти всіх службовців, які отримують зарплату більшу, ніж їх керівники. Цей запит за допомогою елементів прикладів можна сформулювати таким чином: " знайти всіх службовців, чиї керівники є head та отримують зарплату, більшу, ніж s, де s зарплата head " Шаблон відповідного запиту має вигляд:

EMP	ПІБ	Зарплата	Керівник	Відділ
	Р.	$\cdot > \underline{s}$	<u>head</u>	
	<u>head</u>	$\underline{s}$		

Елемент head використовується для зв'язку керівника в першому рядку шаблону та імені в другому рядку, а елемент s використовується для порівняння зарплат. В цьому прикладі результуюча таблиця виявиться такою:

EMP	ПІБ
	Гофман В.Е.

В якості умови вибору записів із таблиць можна використовувати операцію заперечення.

Приклад 9. Відбір з операцією заперечення

Нехай необхідно вивести всі відділи, що продають товари, які не постачаються компанією Pencraft. Цей запит можна перефразувати: “Вивести назви відділів, що продають товари t , такі, що компанія Pencraft не постачає товари t ”.

SALES	ВІДДІЛ	Товар
	P.	t

supply	товар	постачальник
	8 t	Pencraft

Відповідна таблиця для сформульованого запиту має вигляд:

SALES	ВІДДІЛ
	хозтовари
	косметика
	іграшки

У випадках, коли умови відбору записів для вибірки є великими виразами, які незручно або важко задавати в запиті можна використовувати блок умов. За виглядом він нагадує один шаблон з пустим полем та іменем CONDITIONS. Блок умов призначений для запису логічних виразів. Записані в одному шаблоні логічні вирази, в загальному випадку можуть включати в себе операції логічного множення (AND) та логічного додавання (OR).

Приклад 10. Використання блоку умов

Шаблон запиту виводу прізвищ співробітників, чия зарплата складає від 400 до 500, але не дорівнює 460, матиме вигляд:

EMP	ПІБ	Зарплата
	P. <u>Jon</u>	>400
	<u>Jon</u>	<500
	<u>Jon</u>	!=460

Використовуючи блок умов з явним заданням операції AND (&), цей запит можна сформулювати так:

EMP	ПІБ	Зарплата
	P.	<u>s</u>

CONDITIONS
<u>S</u> =(>400&<500&#!#460)

Шаблон запиту виводу прізвищ співробітників, чия зарплата 400, 460 або 520, можна сформулювати так:

EMP	ПІБ	Зарплата
	P. <u>Jon</u>	400
	P. <u>Mak</u>	500
	P. <u>Nik</u>	520

В кожному рядку шаблону використовуються різноманітні елементи прикладів і тому ці умови діють незалежно. За допомогою блока умов, в яких операція OR (символ |) задана явно, цей же запит буде виглядати більш наглядно.

EMP	ПІБ	Зарплата
	P.	<u>s</u>

CONDITIONS
<u>S</u> =(400 500 460)

При запису логічних виразів на QBE можуть застосовуватись **вбудовані функції**, такі як: CNT.(лічильник або кількість), SUM. (сума), AVG. (середнє), MIN. (мінімум), MAX.(максимум), UN. (унікальний) та ALL (всі значення, втому числі ті, що повторюються). Перші 5 є статистичними, а 2 останні визначають чи включати у вибір ці значення.

Приклад 11. Використання функцій

Нехай потрібно вивести назви відділів, в яких працює більше 2 співробітників. Цей запит можна розділити на 3 операції: згрупувати співробітників по відділах, підрахувати число співробітників в кожному відділі та відібрати відділи, в яких працює більше 2 співробітників.

EMP	ПІБ	Відділ
	ALL. <u>Employee</u>	P. <u>Toy</u>

CONDITIONS
CNT.ALL. <u>Employee</u> >2

Конструкція Toy означає операцію **групування**, функція ALL.Employee формує множину всіх імен по кожному з відділів. Запис CNT.ALL.Employee >2 забезпечує перевірку логічної умови. Тут зустрілася нова операція – формування групи (яка записується жирним шрифтом елементу прикладу). Результуюча таблиця матиме вигляд:

EMP	Відділ
	іграшки
	косметика

Інший приклад. Вивести назви відділів, в яких продаються тільки товари зеленого кольору.

SALES	ВІДДІЛ	Товар
	Р. <u>Toy</u>	aLL. lnk

TYPE	ТОВАР	КОЛІР
	aLL. lnk	зелений

Таблиця відповіді для цього запиту буде порожньою.

ЕМР	ПІБ	Зарплата	Керівник	Відділ
	КисельовВ.М.	360	Белкін Б.Н.	госптовари
	Гурський С.І.	320	Томілов А.Н.	іграшки
	Андрєєва Є.А.	400	Петров А.С.	косметика
	Левін П.Г.	440	Петров А.С.	канцтовари
	Носов А.П.	320	Томілов А.Н.	іграшки
	Гофман В.Е.	520	Андрєєва Є.А.	косметика
	Сорокіна Т.В.	340	Андрєєва Є.А.	косметика
	Белкін Б.Н.	360	Петров А.С.	госптовари
	Сьомін С.В.	440	Левін П.Г.	канцтовари
	Григор'єв А.Н.	380	Томілов А.Н.	іграшки
	Томілов А.Н.	400	Петров А.С.	іграшки

SALES	Відділ	Товар
	канцтовари	папір
	госптовари	мило
	канцтовари	олівець
	косметика	помада
	іграшки	літак
	іграшки	машина
	іграшки	лялька
	косметика	парфуми
	канцтовари	чорнила
	госптовари	посуд
	канцтовари	ручка

SUPPLY	ТОВАР	ПОСТАЧАЛЬНИК
	ручка	Pencraft
	папір	Pencraft
	мило	Procter&Gamble
	олівець	Flic
	чорнило	Pencraft
	парфуми	Beautex
	чорнило	Flic
	посуд	Cremco
	помада	Beautex
	літак	Signal
	машина	Signal
	лялька	Signal
	посуд	Flic
	ручка	Beautex
	олівець	Pencraft

TYPE	ТОВАР	КОЛІР	ВАРТІСТЬ
	посуд	білий	30
	помада	червоний	17
	парфуми		42
	ручка	зелений	6
	олівець	синій	2
	чорнило	зелений	4
	чорнило	синій	3
	олівець	червоний	2
	олівець	синій	2